Студент группы ИУ5-21Б

Ерицян С.Г.

Домашнее задание №1

Отчет

Цель работы: разработать программу на языке C# для интерактивного вычисления расстояния Дамерау-Левенштейна между двумя строками и протестировать её работу на различных наборах данных.

Назначение программы: программа предназначена для вычисления расстояния Дамерау-Левенштейна между двумя строками, введенными пользователем. Расстояние Дамерау-Левенштейна — это минимальное количество операций (вставка, удаление, замена или перестановка двух соседних символов), необходимых для преобразования одной строки в другую.

Текст программы:

```
using System;

while (true)

{

   Console.Write("Введите первую строку: ");

   string? str1 = Console.ReadLine();

   // Проверка на выход из программы

   if (str1?.ToLower() == "exit")

   {

      Console.WriteLine("Программа завершена.");

      break;

   }

   Console.Write("Введите вторую строку: ");

   string? str2 = Console.ReadLine();

   Console.WriteLine();

   // Проверка на null (на случай непредвиденных ситуаций)

   if (str1 == null || str2 == null)

   {

      Console.WriteLine("Ошибка: введена пустая строка.");

      continue;

   }

   // Вычисление и вывод расстояния
```

```
    int distance = CalculateDamerauLevenshteinDistance(str1, str2);
    Console.WriteLine($"Результат");
    Console.WriteLine($" \"{str1}\"-> \"{str2}\"");
    Console.WriteLine($" Расстояние Дамерау-Левенштейна: {distance}");
    // Пояснение для пользователя
    Console.WriteLine($" (требуется {distance} операций для преобразования)");
    Console.WriteLine();
}
/// <summary>
/// Вычисление расстояния Дамерау-Левенштейна между двумя строками
/// Реализация алгоритма Вагнера-Фишера
/// </summary>
/// <param name="str1">Первая строка</param>
/// <param name="str2">Вторая строка</param>
/// <returns>Расстояние Дамерау-Левенштейна</returns>
static int CalculateDamerauLevenshteinDistance(string str1, string str2)
{
    // Проверка на null (для безопасности)
    if (str1 == null || str2 == null)
        return-1;
    int len1 = str1.Length;
    int len2 = str2.Length;
    // Если обе строки пустые- расстояние 0
    if (len1 == 0 && len2 == 0)
        return 0;
    // Если одна из строк пустая- расстояние равно длине другой строки
    if (len1 == 0)
        return len2;
    if (len2 == 0)
        return len1;
    // Приводим строки к верхнему регистру для сравнения без учета регистра
    string s1 = str1.ToUpper();
    string s2 = str2.ToUpper();
    //Создаем матрицу размером (len1 + 1) * (len2 + 1)
```

```
    int[,] matrix = new int[len1 + 1, len2 + 1];
    // Инициализация нулевой строки и нулевого столбца
    for (int i = 0; i <= len1; i++)
        matrix[i, 0] = i; // i удалений
    for (int j = 0; j <= len2; j++)
        matrix[0, j] = j; // j удалений
    // Заполнение матрицы
    for (int i = 1; i <= len1; i++)
    {
        for (int j = 1; j <= len2; j++)
        {
            // Проверка на совпадение символов
            // m(s1[i], s2[j])- равно 0 если символы совпадают, иначе 1
            int symbEqual = (s1[i- 1] == s2[j- 1]) ? 0 : 1;
            // Три базовые операции
            int insert = matrix[i, j- 1] + 1; // Добавление символа
            int delete = matrix[i- 1, j] + 1; // Удаление символа
            int replace = matrix[i- 1, j- 1] + symbEqual; // Замена символа
            // Выбираем минимальную стоимость
            matrix[i, j] = Math.Min(Math.Min(insert, delete), replace);
            // Проверка на транспозицию (перестановку соседних символов)
            // Это поправка Дамерау к расстоянию Левенштейна
            if (i > 1 && j > 1 && s1[i- 1] == s2[j- 2] && s1[i- 2] == s2[j- 1])
            {
                // Транспозиция считается за одну операцию
                // symbEqual используется для учета возможной замены после транспозиции
                matrix[i, j] = Math.Min(matrix[i, j], matrix[i- 2, j- 2] + symbEqual);
            }
        }
    }
    // Возвращаем значение в правом нижнем углу матрицы- это и есть искомое расстояние
    return matrix[len1, len2];
}
```

Функция CalculateDamerauLevenshteinDistance: это основная вычислительная функция. Она реализует динамический алгоритм Вагнера-Фишера для нахождения расстояния Дамерау-Левенштейна.

Логика работы:

1. Обработка граничных случаев: если одна из строк пустая, расстояние равно длине другой строки.
2. Приведение к верхнему регистру: чтобы сравнение было регистронезависимым (ToUpper()).
3. Создание матрицы: Таблица размером (len1+1) x (len2+1), где ячейка [i, j] хранит минимальное расстояние между первыми i символами первой строки и первыми j символами второй.
4. Инициализация: Первый столбец заполняется номерами строк (стоимость удалений), первая строка — номерами столбцов (стоимость вставок).
5. Заполнение: для каждой ячейки [i, j] вычисляется стоимость трех операций:
    - Вставка: matrix[i, j-1] + 1
    - Удаление: matrix[i-1, j] + 1
    - Замена: matrix[i-1, j-1] + cost (cost = 0, если символы равны, иначе 1). Выбирается минимальная стоимость.
6. Транспозиция (особенность Дамерау): если текущий символ первой строки равен предыдущему символу второй и наоборот (...AB -> ...BA), учитывается возможность перестановки. Стоимость транспозиции сравнивается с текущим минимумом.
7. Результат: Значение в правой нижней ячейке matrix[len1, len2] является искомым расстоянием.

```
Введите первую строку: примеры
Введите вторую строку: предмет

Результат
 "примеры" -> "предмет"
 Расстояние Дамерау-Левенштейна: 4
 (требуется 4 операций для преобразования)

Введите первую строку: Иванов
Введите вторую строку: Баннво

Результат
 "Иванов" -> "Баннво"
 Расстояние Дамерау-Левенштейна: 4
 (требуется 4 операций для преобразования)
```

Рисунок 1 - Результаты выполнения программы

Вывод: программа корректно реализует алгоритм Дамерау-Левенштейна, позволяя учитывать не только вставки, удаления и замены, но и перестановки соседних символов как одну операцию. Это делает её более точной для практических задач (например, проверка орфографии), чем классический Левенштейн. Код устойчив к пустым строкам и регистру символов, а интерактивный режим с командой exit удобен для тестирования.